

Portfolio Jimin Seo

CONTACT

✉ jn2xxcom@gmail.com

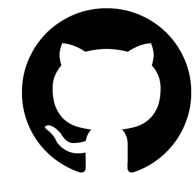
☎ 010-8549-5412

Full-cycle engineer handling everything
from backend development to infra design & operations.

Dev	Java · SpringBoot	Designed & built APIs for 3 projects
	Python · Django	Maintained production backend & features
DB	MySQL · PostgreSQL	Query optimization, SQLD certified
Infra	AWS	AWS SAA certified, operated 3 production projects
	Terraform(IaC)	AWS infrastructure as code
CI/CD	Github Actions	Automated ECR pipelines
	Bash Script	Automated deployment scripting
Others	Docker · Linux · Git	Containerization, server ops, Git-flow

About Me

Jimin Seo



GitHub

@SeoJimin1234



LinkedIn

@jimin-seo-2xx



Blog

<https://seojimin1234.github.io/>

EDUCATION

2021.03 Ewha Womans Univ.
(Major: CSE, Minor: Cyber Security)

2026.08 B.S. Graduation (Expected)

EXPERIENCE

2026.03 - 현재 AWS Cloud Club(Member)

2026.03 - 현재 IT's Time (Dev Club)

2025.08 - 2026.02 Frankfurt UAS (Exchange)

2025.04 - 2025.05 Hanghae99 Kubernetes Course

2024.09 - 2025.02 UMC (Dev Club)

2024.07 - 2024.08 Open Source SW Academy - ArgoCD

2024.02 - 2025.08 EWHA-CHAIN (Vice President)

2023.09 - 2024.02 ECC (Computer Club)

2023.03 - 2024.03 SDP (Tech/PR Lead)

2021.09 - 2024.12 MEDDY (PR Lead)

SKILL

High AWS, SpringBoot, Django,
SQL, Docker

Middle Terraform, Linux, Bash

AWARD

2024.09 WEB2X ConnecTHON
(Grand Prize)

2024.11 Ewha SW Startup Contest
(Encouragement)

2025.05 Korea Univ. Startup Contest
(Encouragement)

LICENSE

Info. Processing Industrial Eng. (Sep 2024)

AWS SAA (Feb 2025)

SQL Developer (Apr 2025)

Network Manager Ch. 2 (Jul 2025)

OPIc English IH (Mar 2026)

Project List

지원자 서지민

 Chattr Large Scale team-chatting service	 Platform Web	 Role Infra Engineer (1 Infra, 2 BE, 1 FE)	 Period 2026.04 - 2026.05
 RePLAN AI failure analysis & To-Do redesign service	 Platform Mobile App (ios, Android)	 Role BE & Infra Engineer (2 BE, 2 FE)	 Period 2026.03 - Ongoing
 TokenUs NFT video-sharing platform	 Platform Web	 Role BE & Infra Engineer (1 BE, 2 FE)	 Period 2024.09 - 2025.11
 SASM Sustainable space curation & map service	 Platform Mobile App, Web	 Role Backend Engineer (4 BE, 4 FE)	 Period 2023.03 - 2024.02



Large Scale **team collaboration chat** service

Core Features

Workspace	Team collaboration hub with multi-channel
Channel	Group chats for select/all workspace members
DM	1:1 private chat

Tech Stack



AWS



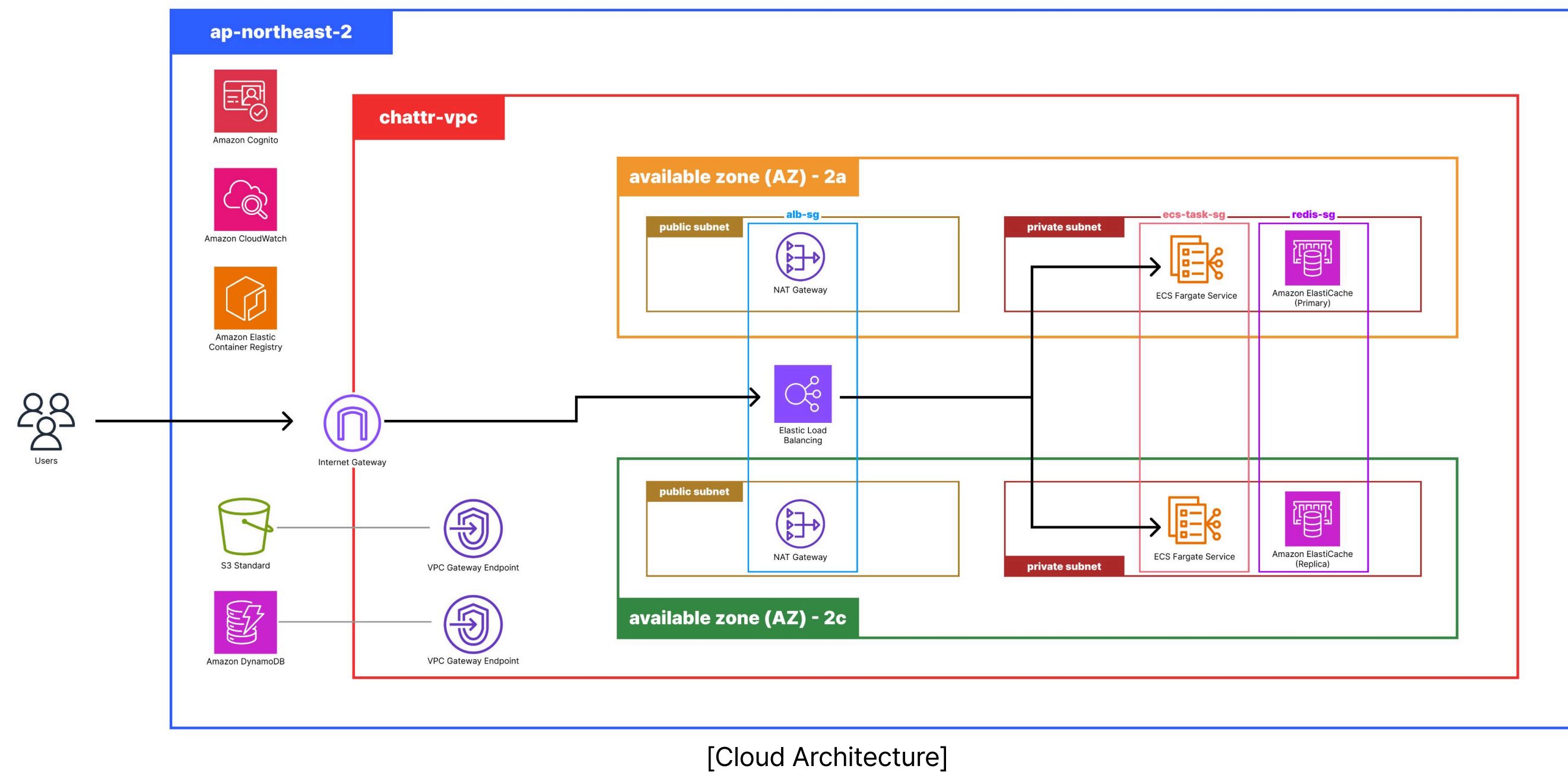
Terraform

Requirements

- ~1,000 active users
- Max 500 concurrent users
- ~50 messages/sec throughput
- 6-month message retention log
- Latency under 2 seconds
- Supports file attachments up to 10MB
- Multi-device sync (Max 3 devices/user)

Contributions

Designed & built AWS cloud infrastructure



Security

Removed direct EC2 access (**Fargate**), isolated **IAM**, enforced **Cognito** JWT, restricted **SG** inbound, set short-lived Presigned URLs, isolated secrets (**SSM**), used **Private Subnets & VPC Endpoints**.

Cost

Implemented **pay-as-you-go** Fargate, DynamoDB **On-Demand**, and **S3 Lifecycle**. Removed NAT Gateway costs via **VPC Endpoints**.

Performance

Multi-instance broadcasting via Redis Pub/Sub, optimized DynamoDB **GSI**, offloaded ECS via **direct S3 uploads**, reduced latency via AWS Backbone.

HA

Multi-AZ subnets + **dual** NAT Gateways, ALB health-check **failover**, NoSQL for **schema flexibility**.

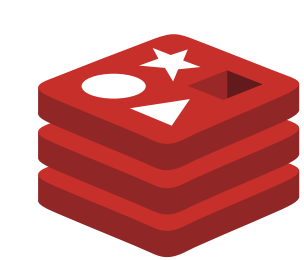
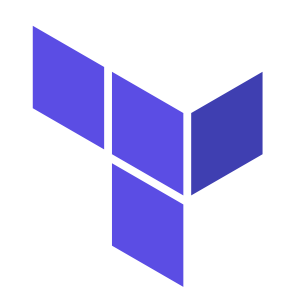


AI-driven To-Do service **analyzing failures** to **generate actionable plans**

Core Features

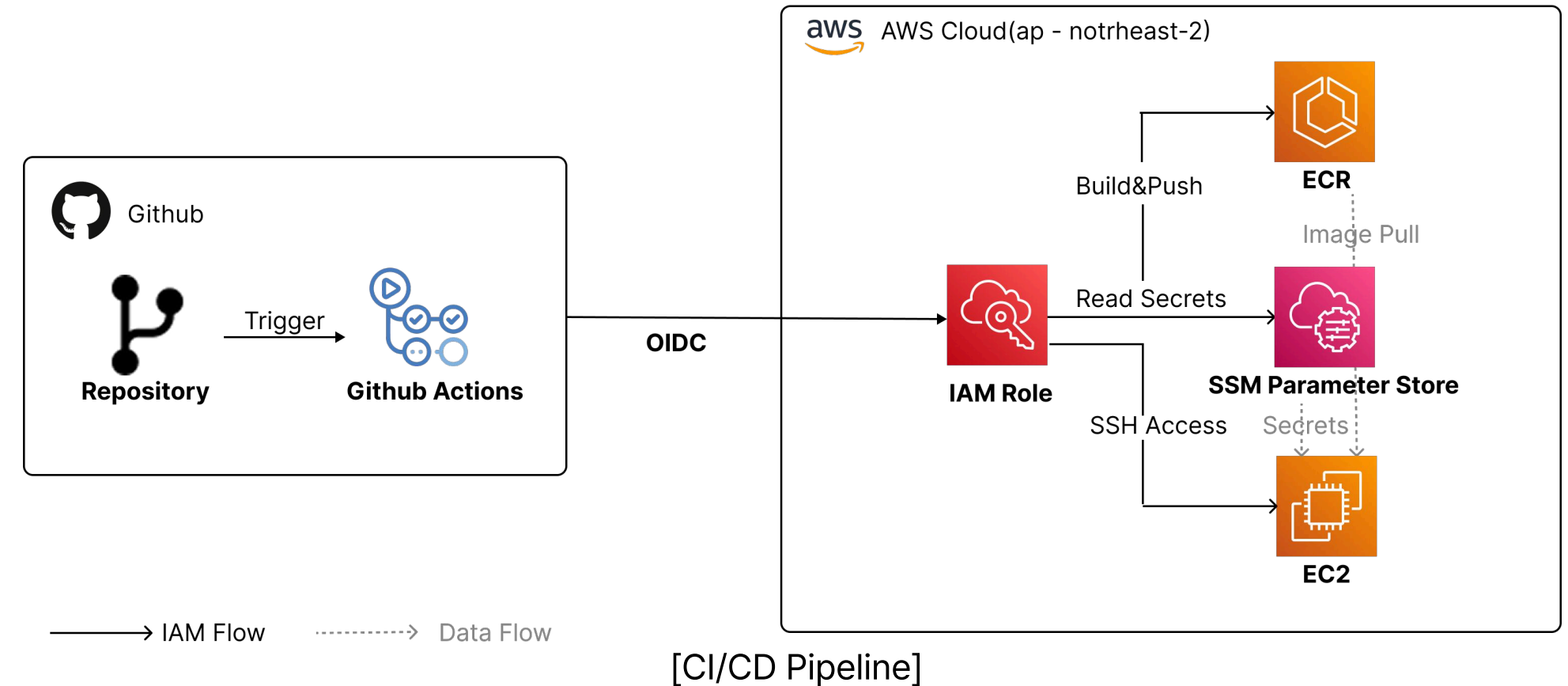
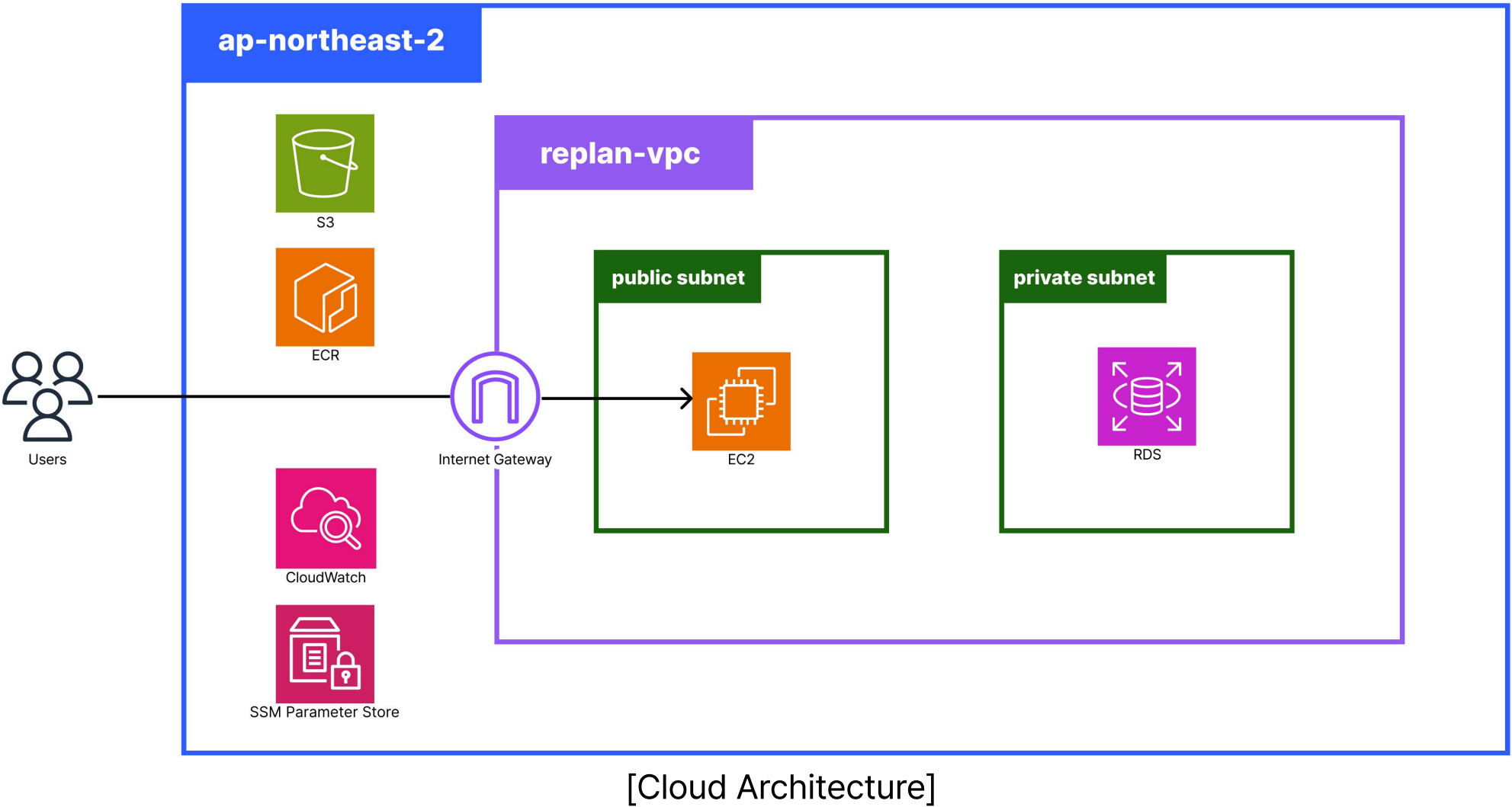
Goal	Recommends/creates customized To-Dos
Home	Manages custom lists
Replan	Provides AI failure analysis & task redesign
Stats	AI feedback based on achievement stats

Tech Stack

	AWS		Docker
	PostgreSQL		SpringBoot
	Redis		Terraform

Contributions

① Infra & CI/CD Pipeline



Security

OIDC authentication (no fixed keys), **IAM** least privilege, **Private Subnet** for DB, and **SSM** Parameter Store for env variables.

CI/CD

Full automation from image build to **ECR** push and SSH app update via **GitHub Actions**.

Ops & Data

Managed 3 containers via **Docker Compose**. Used Redis caching, Nginx reverse proxy, AWS RDS, and **Flyway migration**.

Monitoring

Real-time log/resource metrics collection via **CloudWatch** for instant troubleshooting.

Contributions

② Monthly Statistical Report Batch Job

Bottleneck Analysis

- Per-user processing time: DB aggregation & storage take tens of ms, Gemini API calls take seconds
- Gemini API limit (4,000 RPM) was not reached even with 2 threads
- Actual bottleneck: t2.micro EC2 server specifications



Environmental Constraints

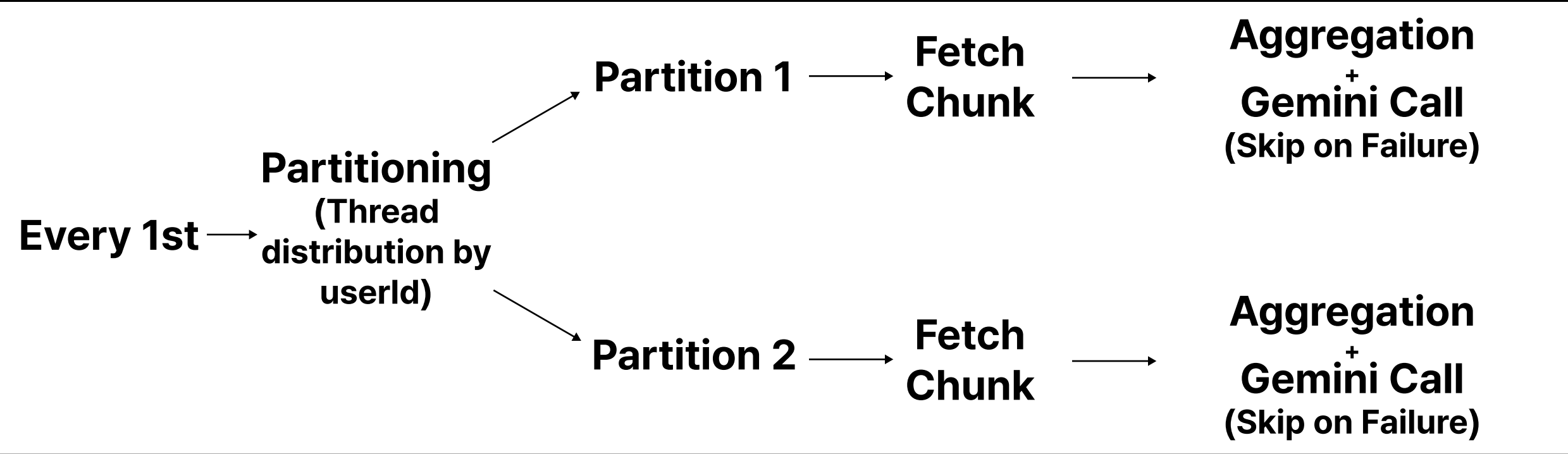
- CPU Credit, HikariCP limits: Excessive burstable instance threads risk CPU credit depletion
- 1GB RAM exhaustion risk: Lack of JVM heap during findAll() full scans



Parameter Decision

- Partition (Thread) Count = 2 • Chunk Size = 50 users

[Parameter Decision Process]



[Batch Job Diagram]

Spring Batch

Chosen for scheduled batch jobs without event triggers; uses existing DB for persistence without external brokers.

Tuning

Identified t2.micro server specs as the real bottleneck (not Gemini API); **optimized thread count and chunk size** accordingly.

Reliability

Isolated errors with a **Skip Policy** to prevent full crashes. Resumes from failure points using Job Repository.

Optimization

Inactive users bypass Gemini API calls to save empty reports directly.



TokenUs

Jimin Seo

Blockchain video-sharing platform **protecting creators** from copyright infringement

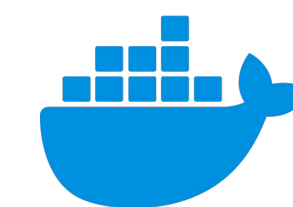
Core Features

Similarity Check	Verifies video uniqueness
NFT Minting	Issues video content as NFTs
NFT Trading	Automates ownership transfer and transaction history
Streaming	Real-time streaming, reactions, and pricing

Tech Stack



AWS



Docker



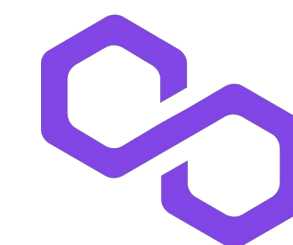
MySQL



SpringBoot



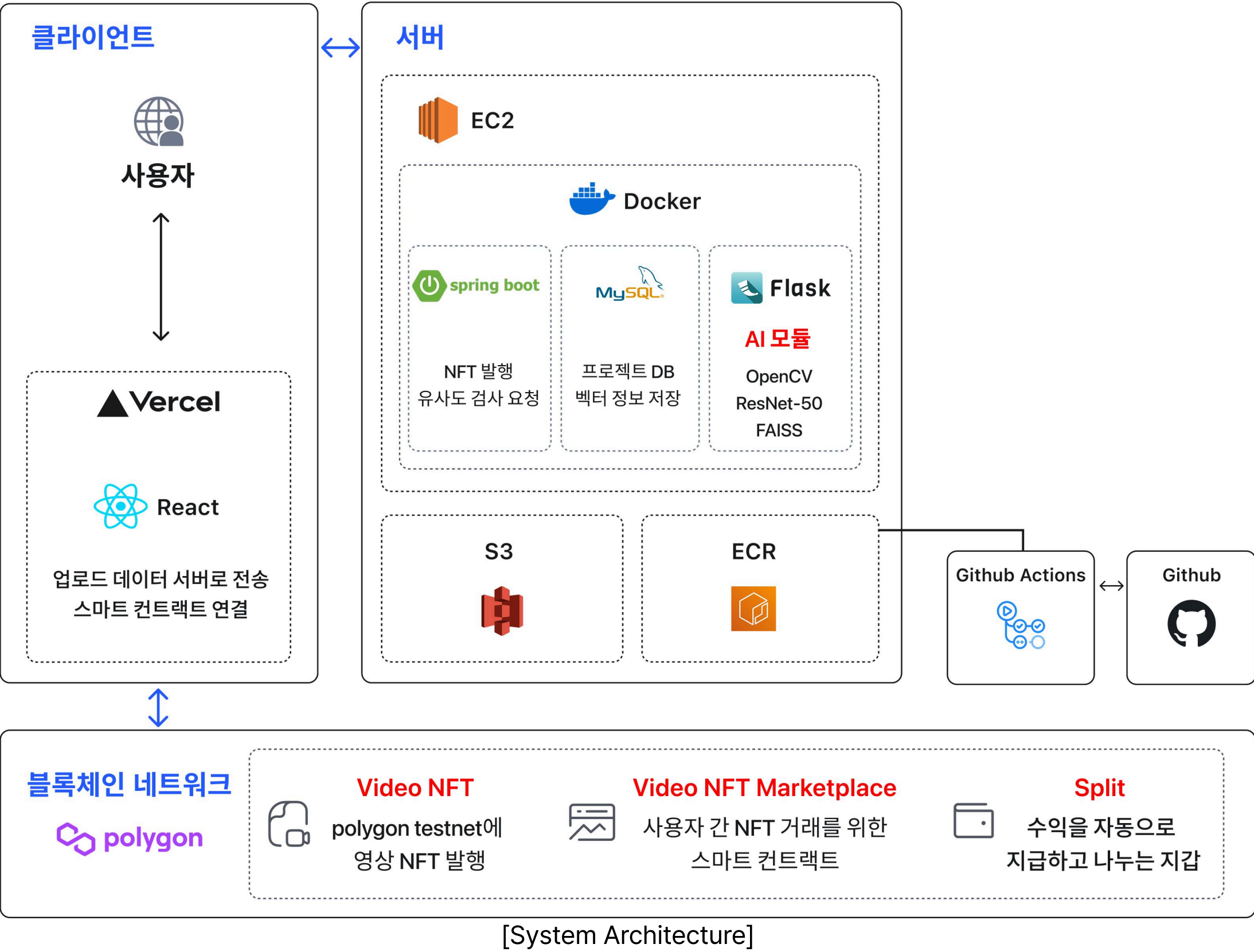
Solidity



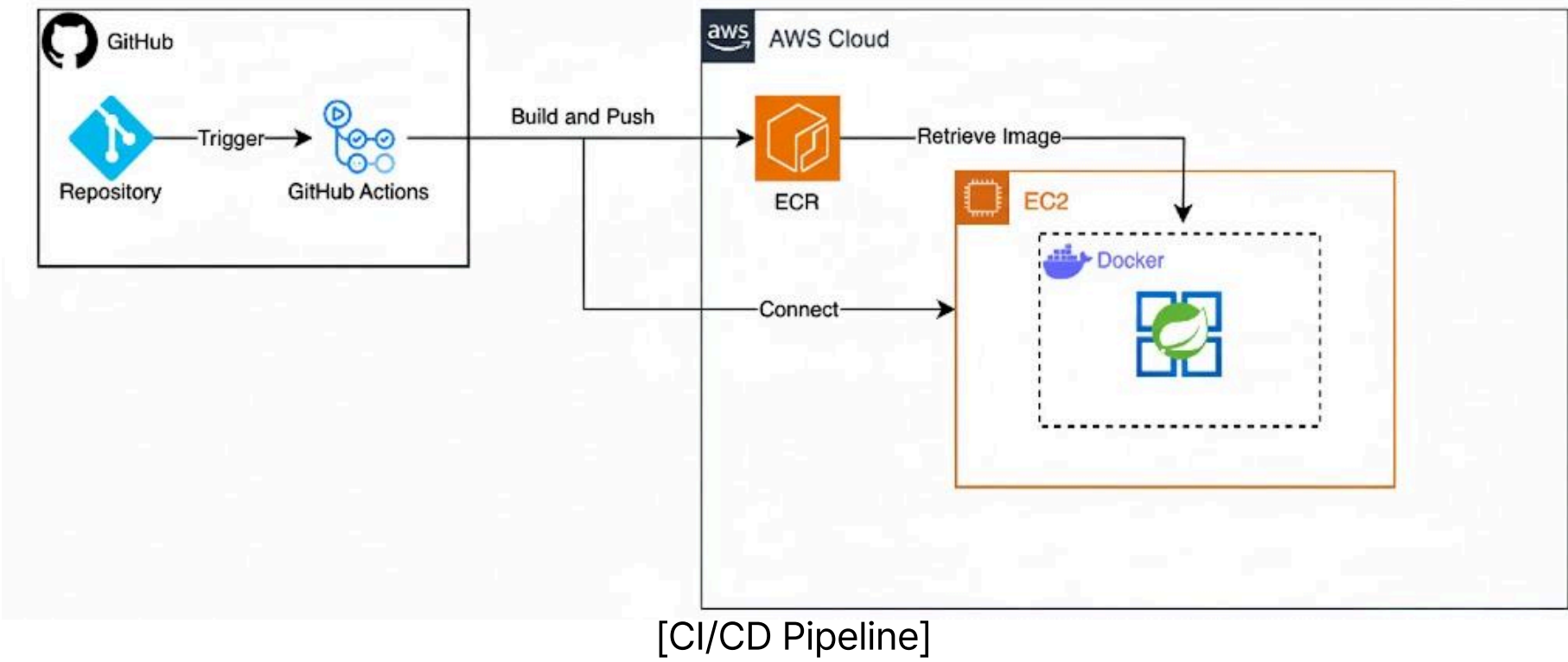
Polygon

Contributions

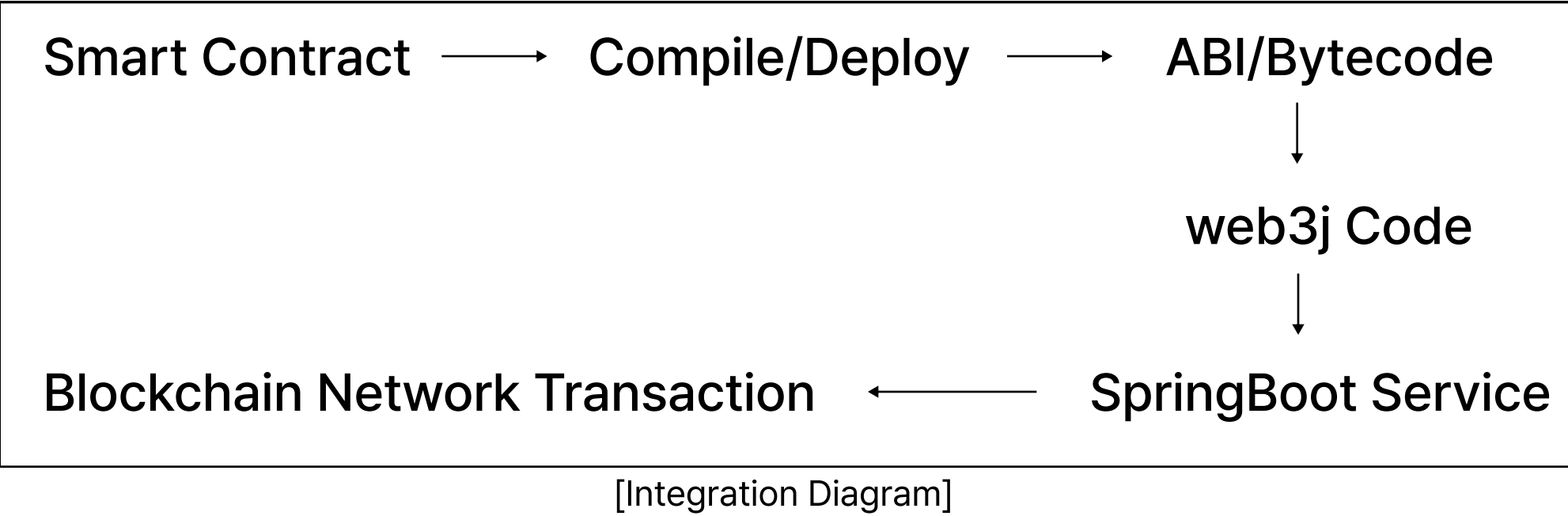
① Architecture



② CI/CD Automation



③ Blockchain Integration



Troubleshooting

Jimin Seo

① Resolved ML Calculation Timeout with WebSocket & Asynchronous Processing

Problem Situation

```
유사도 결과 수신:  
Object {  
  elapsed_time: 48.51  
  max_segment_similarity: 0.99159974  
  message: "Similarity check failed"
```

Video similarity analysis took over 48 seconds on average, **exceeding the synchronous request handling limits** of the Spring Boot API server.

Clients **failed to receive results**, causing **disconnected** connections.

Cause & Phenomenon Analysis

[Root Cause: ML Calculation Bottleneck]

Spring Boot synchronously waited for results on a **heavy**, real-time response-challenging video data **processing** task.

[Phenomenon: Resource Waste]

Clients had to maintain connections indefinitely to get a response, **lowering server resource efficiency**.

Resolution

[WebSocket Adoption]

Replaced one-time HTTP connections with a **persistent connection channel** between the client and server.

[Asynchronous Processing & Result Push]

Returned "Receipt Complete" instantly to prevent timeouts, and **pushed** results immediately upon calculation completion.

Insights

Realized the importance of **asynchronous processing** and **resource efficiency**.

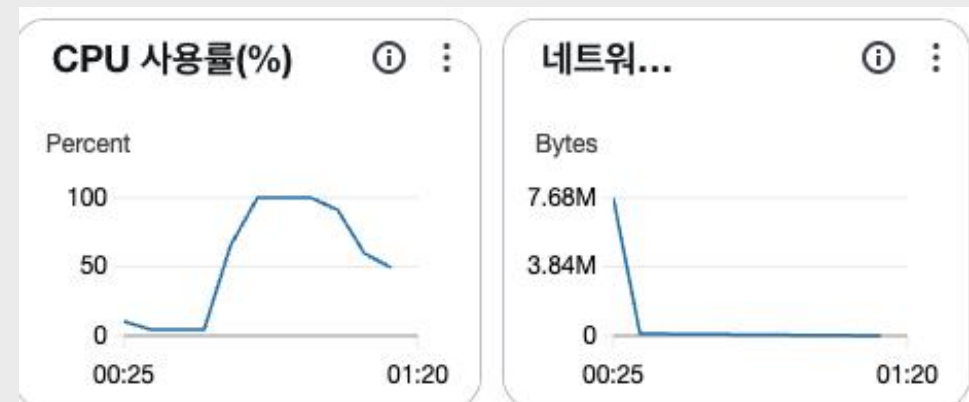
Since this project requires only unidirectional push, transitioning to **SSE** (Server-Sent-Events) could achieve a lighter implementation without external dependencies.

Troubleshooting

Jimin Seo

② Resolved Server Downtime (Reduced CPU Usage from 99% to Under 50%)

Problem Situation



Right after deploying the Spring Boot container to EC2, CPU usage exceeded 99% and memory was exhausted, **causing EC2 failure**.

MySQL and Flask **containers on the same host crashed sequentially**.

Cause & Phenomenon Analysis

[Initial Boot Load]

High CPU/Memory **spike during startup** due to dependency loading and JPA initialization.

[Resource Exhaustion & Bottleneck]

Memory shortage in Free Tier t2.micro (1GB RAM) led to CPU bottleneck and freeze.

[Monopoly & Cascading Failure]

Lack of container isolation allowed resource **monopoly**, depriving other containers.

Resolution

Deployment Optimization

- **JPA Config**: Set ddl-auto: none to reduce boot load.
- **Resource Limit**: Restricted Spring Boot container to CPU 0.5 and Memory 512M.

Memory Improvement

Configured 2GB **Swap space** to absorb startup memory peaks.

Insights

Realized the necessity of **resource management and optimization** in constrained environments.

Understood that container-level resource limits are vital to **prevent a single container from crashing the entire shared host**.



Mapping and **curation** service for **sustainable** leisure spaces

Core Features

Place(Map)	Interactive map of sustainable locations
Forest	Community space for user thoughts
Story	Curated articles on specific spaces
MyPage	Saved likes, reviews, and custom user logs

Tech Stack



MySQL



AWS



Django



Docker

Contributions

① Feature Design & Implementation

Designed and implemented the **MyPage API** while maintaining system consistency.

② Production Issue & Bug Fixes

Conducted self-exploratory **testing** prior to launch to identify and resolve edge-case bugs.

③ Legacy Code Improvement

Maintained consistency with legacy code by adhering to common product conventions and design patterns.

에러 목록

• 에러 작성 시 최대한 구체적으로 작성 부탁드립니다!
(사진 캡처, 발생 위치와 구체적인 현상 작성)

표

↑ 우선순위 | 해결: 체크 표시됨 | + 필터

우선순위	테크 메모	발생 위치	해결	Aa 이름
1순위		MyPage	✓	유저 프
1순위		Map	✓	맵에서 에 맞춰
1순위	BE 구현 완료(#544)	etc	✓	통합 검
1순위		Map	✓	리뷰에
1순위		Story	✓	스토리
1순위		MyPage	✓	유저 정
2순위		Forest	✓	Board 튼을 누 X 오류
2순위		Map	✓	홈에서 당하는 목록들

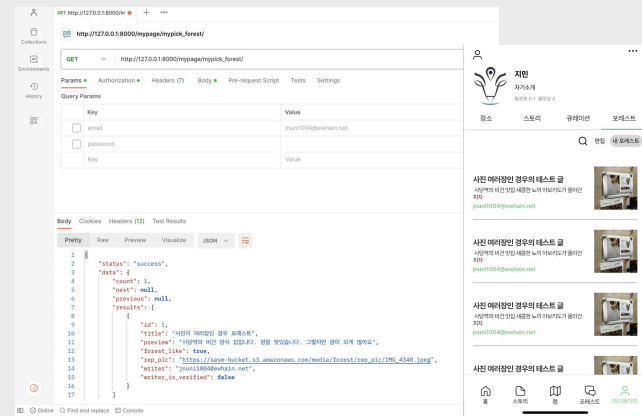
- ☐ [#472 fix: Add SamePlaceStory informations](#)
#481 by SeoJimin1234 was merged on Jul 30, 2023 • Approved
- ☐ [fix: Remove duplication at mypick_forest](#)
#487 by SeoJimin1234 was merged on Aug 1, 2023 • Approved
- ☐ [#487 fix: Remove my_forest Duplication](#)
#488 by SeoJimin1234 was merged on Aug 2, 2023 • Approved
- ☐ [fix: Add Nickname in Mypage/Forest](#)
#496 by SeoJimin1234 was merged on Aug 9, 2023 • Approved
- ☐ [fix: Modify Story vegan_category Selector](#)
#501 by SeoJimin1234 was merged on Aug 14, 2023 • Approved
- ☐ [fix: Return Nickname at Curation List](#)
#506 by SeoJimin1234 was merged on Aug 18, 2023 • Approved
- ☐ [fix: Replace preview with Summary in Story Search](#)
#509 by SeoJimin1234 was merged on Aug 20, 2023 • Approved
- ☐ [#520 fix: User semi Cateory Blank True](#)
#526 by SeoJimin1234 was merged on Sep 23, 2023 • Approved
- ☐ [#540 fix: Curation Order set default](#)
#541 by SeoJimin1234 was merged on Oct 14, 2023 • Approved

Troubleshooting

Jimin Seo

Resolved Duplicate Object Returns → Improved API Response Speed by 20%

Problem Situation



Querying posts with hashtags returned **duplicate** identical objects (proportional to hashtag count).

No duplicate DB records, but identical IDs repeated in GET results.

Cause & Phenomenon Analysis

[SQL Join Behavior]

Many-to-Many hashtag filtering triggered **internal SQL JOINS**, generating N rows for N hashtags at the DB level.

[N+1 Problem]

Querying associated hashtags for each duplicate post triggered extra queries, causing an N+1 issue.

Resolution

[distinct() - Deduplication]

Removed duplicate rows at SQL level, reducing DB transfer load and server object overhead.

[prefetch_related() - N+1 Fix]

Pre-fetched hashtag data, reducing multi-row queries down to just 2 queries.

Insights

Diagnosed how M2M filtering causes SQL Joins and N+1 queries, deepening understanding of **ORM mechanics**.

Confirmed that reducing query count alone **boosts response speed** by 20%, realizing its direct impact on performance.